

Rapport Technique

Conception d'un Agent IA Assistant & Système RAG Intelligent

Option 4 : Guide Touristique et Culturel Intelligent -
Destination Burkina Faso

Lien Github : [https://github.com/techinnov6-debug/RAG tourisme BF.git](https://github.com/techinnov6-debug/RAG_tourisme_BF.git)

Lien streamlit : <https://ragtourismebf-xjix4k5vcmbzqgktbpwzer.streamlit.app/>

Cours : Data Science

Classe : Master 1 IFOAD-UJKZ

Enseignant : Dr Delwende D. Arthur Sawadogo

Groupe :

ZERBO Soukrane Zanhira

SANA Ali

SAWADOGO Aboubacary

Date : 12/07/2026

1. Introduction et objectif

Ce projet consiste à concevoir un agent conversationnel intelligent capable de répondre à des questions sur le tourisme, la culture et les formalités de voyage au Burkina Faso, en s'appuyant sur un système de génération augmentée par récupération (RAG - Retrieval-Augmented Generation). Plutôt que de reposer sur les connaissances générales d'un grand modèle de langage, l'agent recherche d'abord l'information pertinente dans une base documentaire constituée de sources réelles, avant de générer une réponse strictement fondée sur ce contexte.

L'objectif pédagogique est double : dépasser le simple chatbot conversationnel en intégrant une recherche documentaire réelle, et limiter le risque d'hallucination en permettant à l'agent de reconnaître explicitement les limites de ses connaissances.

2. Architecture du système

Le système est structuré en deux flux distincts : un flux d'ingestion hors-ligne, qui alimente une base vectorielle à partir des sources collectées, et un flux de requête en ligne, déclenché à chaque question de l'utilisateur.

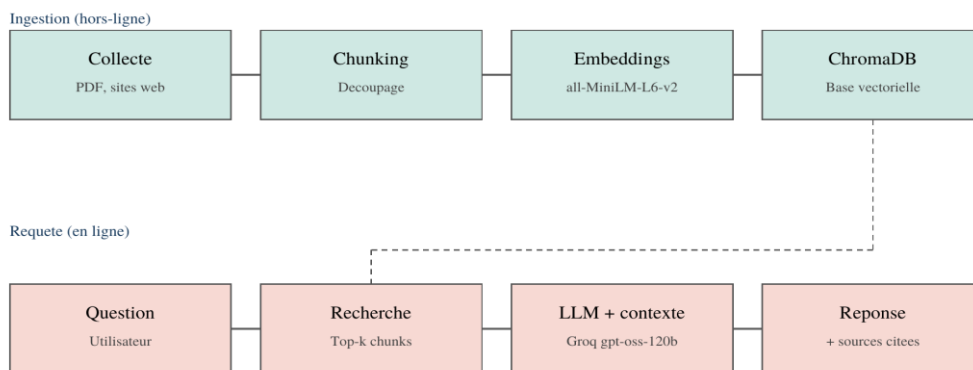


Figure 1 - Flux d'ingestion (haut) et flux de requête (bas) du système RAG.

Flux d'ingestion : les documents sources (pages web scrapées, PDF officiels) sont collectés, nettoyés, découpés en segments (chunks), transformés en vecteurs numériques (embeddings), puis stockés dans une base vectorielle ChromaDB.

Flux de requête : la question de l'utilisateur est transformée en vecteur avec le même modèle d'embeddings, comparée aux vecteurs stockés pour identifier les chunks les plus pertinents (recherche par similarité), puis ces chunks sont transmis en contexte à un grand modèle de langage (LLM) via l'API Groq, qui génère la réponse finale.

3. Méthodologie

3.1 Collecte des données

La collecte combine le scraping de sites web réels liés au tourisme burkinabè (Faso Tourisme, Discover, FESPACO, SIAO, plateformes de voyage, pages Wikipedia sur les sites culturels) et l'extraction de texte depuis des PDF officiels. Le scraper explore chaque site en largeur (parcours BFS), en restant sur le même domaine, jusqu'à une profondeur et un nombre de pages maximum configurables par source. Pour les plateformes mondiales (Booking.com, Evaneos, Partir.com), un filtre par mots-clés d'URL restreint le parcours aux pages relatives au Burkina Faso plutôt que de limiter arbitrairement le nombre de pages.

Deux mécanismes de filtrage garantissent la qualité du corpus : le respect du fichier robots.txt de chaque domaine, et une détection de la langue du contenu extrait (le corpus étant destiné à rester exclusivement en français), en complément d'un filtrage par segments d'URL (ex. /en/).

3.2 Chunking et vectorisation

Les textes collectés sont découpés en segments d'environ 500 caractères, avec un chevauchement de 80 caractères entre segments consécutifs, selon une stratégie récursive par séparateurs (paragraphe, phrase, mot) qui privilégie les coupures naturelles.

Le modèle d'embeddings retenu est **all-MiniLM-L6-v2** (Hugging Face / sentence-transformers), léger et rapide. Les vecteurs sont normalisés (norme 1) avant indexation, afin que la distance L2 utilisée par ChromaDB reste bornée et interprétable (une distance de 0 correspond à des vecteurs identiques, une distance de 4 à des vecteurs opposés). Les vecteurs sont stockés dans une collection ChromaDB persistée localement, reconstruite intégralement à chaque exécution du script de vectorisation pour éviter toute duplication de chunks lors des relances.

3.3 Logique de l'agent

À chaque question, les cinq chunks les plus proches (top-k = 5) sont récupérés. Si la distance du meilleur résultat dépasse un seuil configuré, le système considère qu'aucune information pertinente n'est disponible et répond explicitement qu'il ne dispose pas de cette information, sans solliciter le LLM -- ce mécanisme constitue la principale protection contre l'hallucination.

Le modèle de langage utilisé est **openai/gpt-oss-120b**, servi via l'API Groq. Le choix initial s'était porté sur Llama 3 (recommandation de l'énoncé), mais Groq a annoncé le 17 juin 2026 la dépréciation des modèles llama-3.3-70b-versatile et llama-3.1-8b-instant au profit de la famille openai/gpt-oss, ce qui a motivé ce changement pour garantir la disponibilité du modèle jusqu'à la remise du projet. Le prompt système impose une réponse exclusivement en français, fondée uniquement sur le contexte fourni, avec citation des sources utilisées.

3.4 Interface utilisateur

L'interface est développée avec Streamlit (composant `st.chat_message`), avec historique de conversation, affichage des sources consultées pour chaque réponse, et indicateur d'état de la configuration (clé API).

4. Évaluation du système

Conformément à l'énoncé, le système est évalué selon deux axes, à l'aide d'un jeu de treize questions de test :

- **Pertinence de la recherche documentaire** : pour huit questions dont la catégorie de réponse attendue est connue (site culturel, événement, formalités, institution), on vérifie si cette catégorie figure bien parmi les documents récupérés dans le top-k.
- **Taux d'hallucination** : pour cinq questions volontairement hors périmètre (sans rapport avec le Burkina Faso), on vérifie que l'agent répond bien qu'il ne dispose pas de l'information, plutôt que d'inventer une réponse.

Le script d'évaluation calcule également, pour chaque catégorie de questions, la plage de distances observées, ce qui permet de calibrer empiriquement le seuil de détection du hors-périmètre plutôt que de se fier à une valeur arbitraire. Les résultats détaillés (question par question) sont exportés dans un fichier CSV. [Synthétiser ici les résultats obtenus lors de l'exécution réelle : taux de pertinence et taux d'hallucination mesurés, avant la remise.]

5. Limites et perspectives

- **Contenu chargé en JavaScript** : les plateformes Booking.com, Evaneos.fr et Partir.com affichent l'essentiel de leur contenu (offres, hébergements) via JavaScript ; le scraper actuel (requests + BeautifulSoup) ne récupère que le HTML statique. Un outil de rendu (Selenium, Playwright) serait nécessaire pour exploiter pleinement ces sources.
- **Formalités de voyage incomplètes** : aucune source officielle dédiée aux formalités de visa n'a encore été intégrée de façon exhaustive ; seules des pages généralistes sont utilisées à ce stade.
- **Seuil de détection du hors-périmètre** : la valeur configurée reste une estimation de départ, à affiner avec davantage de questions de test et, idéalement, un jugement humain sur un échantillon de réponses.
- **Dette technique assumée** : chaque phase du projet dispose de son propre fichier de configuration et de son propre script principal, afin de garder chaque étape compréhensible indépendamment durant le développement. Une consolidation partielle (valeurs partagées critiques) a été réalisée en fin de projet.

- **Ajout des images aux réponses :** Pour le moment, nous n'avons pas intégré la rdu contenu visuel aux réponses fournies par l'agent

-

6. Conclusion

Le système développé répond à l'objectif fixé : un agent capable de rechercher une information réelle avant de répondre, et de reconnaître les limites de ses connaissances plutôt que d'halluciner. Les principales pistes d'amélioration identifiées (modèle d'embeddings multilingue, gestion des sites dynamiques, calibration fine du seuil de pertinence) constituent des prolongements naturels au-delà du périmètre de ce projet académique.